PROJECT ATLAS — INTERFACE CONTROL DOCUMENT

ATLAS-ICD-001
LED Display Interface Control Document

Master Flightline Clock — 6-Row LED Display Interface Specification

DOCUMENT NUMBER	ATLAS-ICD-001
REVISION	1.0 — Initial Release
ISSUE DATE	3 July 2026
STATUS	APPROVED FOR USE
CLASSIFICATION	Project Confidential
AUTHOR	ATLAS Engineering Team
HARDWARE TARGET	P5 Indoor / P10 Outdoor LED Matrix
SOFTWARE VERSION	v1.5.0 (Simulator) / HAL v1.0 (Future)

UNCONTROLLED WHEN PRINTED

Document Control & Approval Record

Revision History

Table DC-1 Revision History

Rev	Date	Description	Author	Approved By
1.0	2026-07-03	Initial release — modular ICD, simulator backend and HAL architecture defined; hardware-specific sections marked TBD pending procurement	ATLAS Engineering	Project Lead

Document Purpose and Status

This document defines the interface between the ATLAS Master Flightline Clock software system and the physical LED display hardware. It covers physical panel specifications, signal-level electrical requirements, communication protocols, the Hardware Abstraction Layer (HAL) software architecture, and integration requirements.

Sections marked TBD are placeholders that require specific hardware procurement decisions before they can be completed. Sections marked CURRENT reflect the as-built simulator implementation. Sections marked FUTURE define the target physical hardware integration architecture.

Applicable Documents and Standards

Table DC-2 Reference Documents

Reference	Title	Version
ATLAS-SDS-001	ATLAS Software Design Specification	Current
ATLAS-SHD-001	ATLAS System Handover Dossier	2026-07-03
HUB75 Spec	HUB75 RGB LED Matrix Interface Specification	Community Rev D
IEC 60950-1	Information Technology Equipment — Safety	Ed. 2
CISPR 32	Multimedia Equipment EMC — Emission Requirements	Ed. 3
AS/NZS 3820	Essential Requirements for Electrical Equipment	1998

© ATLAS Project 2026

Table of Contents

1. Executive Summary 4

2. System Overview 5

2.1 Architecture Context 5

2.2 Data Flow 6

2.3 Six-Row Display Strategy 6

3. Display Hardware Specifications 7

3.1 LED Panel Options 7

3.2 Resolution and Layout Configuration 8

3.3 Viewing Distance and Brightness 9

4. Interface Protocols 9

4.1 HUB75 RGB Matrix Protocol 9

4.2 Alternative Protocols 9

4.3 Protocol Selection Criteria 10

5. Signal Definitions 10

5.1 HUB75 Connector Pinout 11

5.2 Timing Diagrams 12

5.3 Data Frame Format 13

5.4 Row Addressing for 6-Row Configuration 13

6. Hardware Abstraction Layer (HAL) 14

6.1 Abstract Interface Design 14

6.2 Simulator Backend (Current) 16

6.3 Physical Hardware Backend (Future) 16

6.4 Backend Selection Mechanism 17

7. Integration Requirements 18

8. Testing and Validation 19

Appendix A — Glossary of Terms 21

Appendix B — Reference Documents 23

Appendix C — Revision History 23

© ATLAS Project 2026

1. Executive Summary

This Interface Control Document (ICD) defines the complete interface specification between the ATLAS Master Flightline Clock software system and the physical LED matrix display hardware used at competitive aviation events. The document governs all aspects of the display interface: physical panel selection and mounting, electrical signal definitions, communication protocols, software abstraction architecture, and integration requirements.

The ATLAS system currently operates in a software-simulator mode, where `AtlasLEDSimulator` renders the 6-row display layout in a Qt6 window on the host Mac Mini M4. This ICD establishes the architectural framework and technical specifications needed to transition from the current simulator to physical LED panel hardware, without requiring changes to the core timing engine or operator console.

Scope of Interface Control

This ICD has authority over the following interface boundaries:

- **Software-to-Hardware boundary:** The `DisplayBackend` abstract class defines the contract between the AtlasEngine and any physical or simulated display hardware.
- **Electrical interface:** HUB75 signal levels, timing, connector pinout, and power distribution between the host controller and LED panels.
- **Mechanical interface:** Panel dimensions, mounting provisions, and cabling topology.
- **Operational interface:** Brightness control, on/off sequencing, and fault behaviour during heat operations.

> **NOTE — MODULAR ICD APPROACH**
>
> Sections describing electrical pin-level specifications and mechanical dimensions are currently defined generically for the P5/P10 panel family. Hardware-specific values (exact panel model number, controller board, wiring harness part numbers) shall be inserted in a Rev 1.1 update once hardware procurement is finalised.

Key Design Decisions Codified in This Document

1. The 6-row display layout is the normative rendering target for all physical hardware; it is not a simulator artefact.
2. The Hardware Abstraction Layer (HAL) isolates all hardware-specific code; no HAL code shall appear in the engine, models, or operator console modules.
3. HUB75 is the primary protocol for direct-drive Raspberry Pi / FPGA controller implementations. Artnet DMX and Serial/UART are documented as secondary options for network-based or embedded controller alternatives.

© ATLAS Project 2026

4. Brightness control (10–100%) is a first-class operational requirement and must be implementable on all backends without restarting the application.

2. System Overview

ATLAS (Automated Timing & LED Announcement System) is a Master Flightline Clock designed for competitive aviation events such as Race to the Sky and similar airshow circuits. It provides a single authoritative time reference for track operations, driving both the operator console (internal) and a large-format public LED display (external, visible to pilots and spectators).

2.1 Architecture Context

The system runs on a Mac Mini M4 and implements a strict Model-View-Controller (MVC) architecture. The display subsystem is intentionally isolated from the timing engine via a well-defined data model boundary. Figure 2-1 illustrates the top-level architecture and the position of the LED display interface.

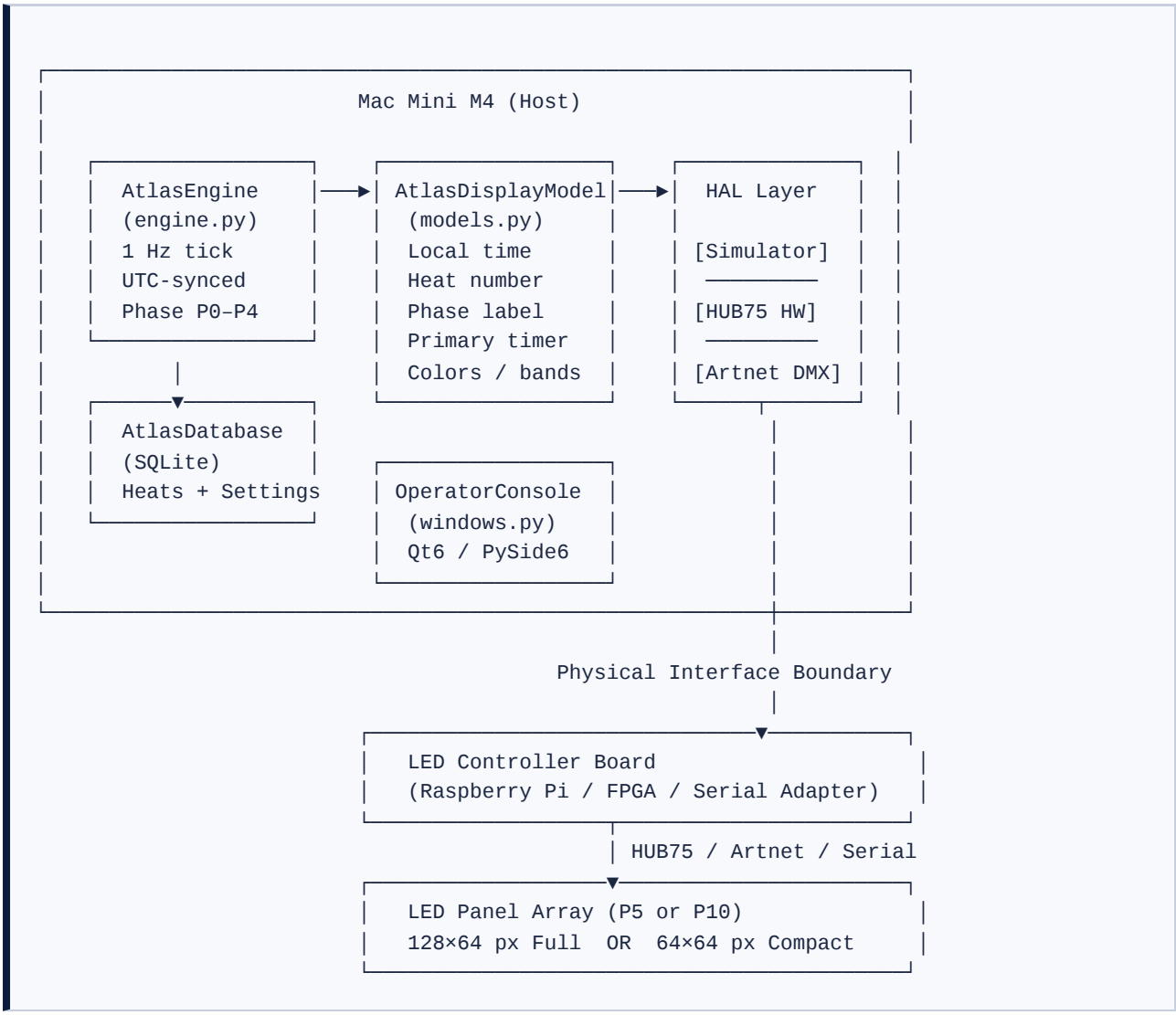


Figure 2-1 ATLAS System Architecture and Display Interface Boundary

© ATLAS Project 2026

2.2 Data Flow

The `AtlasEngine` emits an `AtlasDisplayModel` object once per second via the Qt signal `model_updated`. The HAL backend receives this model and is responsible for translating it into the frame buffer format required by the physical hardware. Table 2-1 defines the fields of `AtlasDisplayModel` and their display mappings.

Table 2-1 AtlasDisplayModel Field Definitions

Field	Type	Example	Display Row	Color
`local_time`	str	"14:32:07"	Row 1	Cyan (#00FFFF)
`heat_number`	str	"03"	Row 2 (prefix H)	Yellow (#FFFF00)
`thermalling_dir`	str	"LEFT" / "RIGHT"	Row 2 (suffix)	Yellow (#FFFF00)
`planned_open`	str	"14:30"	Row 3 (prefix O)	Gray (#AAAAAA)
`planned_close`	str	"14:35"	Row 3 (prefix C) / Row 4 compact	Gray (#AAAAAA)
`primary_timer_label`	str	"TRACK OPENS IN"	Row 4 (full) / Row 5 (compact)	Phase-dependent
`primary_timer`	str	"04:52"	Row 5 (full 2× scale) / Row 6 (compact)	Phase-dependent
`primary_timer_color`	str	"#FFFF00"	Rows 4–5	Model-defined
`band_color`	str	"#555500"	Row 6 (2 px status band)	Model-defined
`is_armed`	bool	True	All rows — gate for live data	N/A

2.3 Six-Row Display Strategy

The display is partitioned into six logical rows regardless of the physical pixel resolution selected. The renderer adapts font scale and text abbreviation to fit both the Full (128×64) and Compact (64×64) profiles. This two-profile strategy is a core ATLAS design requirement and must be preserved by any physical hardware backend.

Table 2-2 Six-Row Layout — Full Mode (128×64)

Row	Content	Y Origin (px)	Font Scale	Color
1	Local Time (HH:MM:SS)	2	1× (5×7)	Cyan

© ATLAS Project 2026

2	Heat number and thermalling direction	12	1× (5×7)	Yellow
3	Track open / close window	22	1× (5×7)	Gray
4	Phase label (TRACK OPENS IN / ENTER NOW / HEAT TIME REMAINING)	35	1× (5×7)	Phase color
5	Primary countdown timer MM:SS	45	2× (scaled)	Phase color
6	Status band (2 px solid stripe)	62	—	Band color

Table 2-3 Six-Row Layout — Compact Mode (64×64)

Row	Content	Y Origin (px)	Alignment	Color
1	Local Time	2	Centered	Cyan
2	H<num> L/R	12	Centered	Yellow
3	O HH:MM (open time)	22	Centered	Gray
4	C HH:MM (close time)	32	Centered	Gray
5	Abbreviated phase label (OPEN IN / ENTER / HEAT LEFT)	42	Centered	Phase color
6	Primary timer MM:SS	53	Centered	Phase color

The threshold for compact mode selection is `display_width ≤ 96 px`, implemented as a comparison at simulator initialisation. Physical hardware backends shall apply the same threshold.

3. Display Hardware Specifications

3.1 LED Panel Options

Two LED panel families are approved for use with the ATLAS display system. Selection between P5 and P10 is driven by the installation environment (indoor versus outdoor) and the required viewing distance. Both panel types use the same HUB75 electrical interface and are therefore interchangeable from a software perspective.

Table 3-1 LED Panel Comparison — P5 Indoor vs. P10 Outdoor

Parameter	P5 Indoor	P10 Outdoor
Pixel Pitch	5 mm	10 mm
Module Size (W × H)	320 × 160 mm	320 × 160 mm

© ATLAS Project 2026

Module Resolution	64 × 32 px	32 × 16 px
LED Type	SMD 2121	SMD 3535 / DIP 346
Peak Brightness	800–1,200 cd/m²	6,000–8,000 cd/m²
Viewing Angle (H × V)	120° × 60°	120° × 60°
IP Rating	IP31 (indoor)	IP65 (outdoor)
Operating Temperature	-10 °C to +45 °C	-20 °C to +55 °C
Power Consumption (peak)	≈ 80 W/m²	≈ 200 W/m²
Refresh Rate	≥ 1,920 Hz	≥ 960 Hz
Colour Depth	16-bit per channel (48-bit RGB)	16-bit per channel (48-bit RGB)
Connector	HUB75E (2 × 8 IDC)	HUB75 (2 × 8 IDC)
Recommended Use	Indoor timing tower, hangar display	Outdoor flight line, grandstand

3.2 Resolution and Layout Configuration

The ATLAS display requires a minimum resolution of 64×64 pixels (Compact profile) and recommends 128×64 pixels (Full profile) for optimal readability of the 5×7 bitmap font. Table 3-2 defines the panel module counts required for each profile.

Table 3-2 Panel Array Configuration

Profile	Resolution	P5 Modules (W × H)	P10 Modules (W × H)	Physical Size (P5)	Physical Size (P10)
Compact	64 × 64 px	1 × 2	2 × 4	320 × 320 mm	640 × 640 mm
Full (recommended)	128 × 64 px	2 × 2	4 × 4	640 × 320 mm	1,280 × 640 mm
Extended (optional)	256 × 64 px	4 × 2	8 × 4	1,280 × 320 mm	2,560 × 640 mm

The `config.py` module currently computes panel resolution from `panels_wide × 32` (width) and `panels_high × 16` (height), consistent with P10 module geometry. For P5 panels, the multiplier changes to `panels_wide × 64` (width) and `panels_high × 32` (height). This parameter shall be made configurable in the database settings table under keys `px_per_module_wide` and `px_per_module_high` in a future software revision.

© ATLAS Project 2026

3.3 Viewing Distance and Brightness

Optimal viewing distance for an LED display is a function of pixel pitch. A practical rule is that the minimum comfortable viewing distance equals approximately 1,000 × pixel pitch (in the same units). Table 3-3 provides viewing distance ranges for both panel types.

Table 3-3 Viewing Distance and Brightness Requirements

Panel	Min. Distance	Optimal Distance	Max. Distance	Daytime Brightness	Night Brightness
P5 Indoor	4 m	8–20 m	30 m	100% (≈1,000 cd/m²)	10–30%
P10 Outdoor	8 m	15–50 m	80 m	100% (≈7,000 cd/m²)	20–40%

The ATLAS brightness control slider (10–100%, five labeled steps: Night / Overcast / Normal / Bright Day / Full Sun) maps directly to these brightness requirements. The software brightness value is transmitted to the backend and must be applied uniformly to all LEDs in the panel array, without per-channel correction at the software layer.

4. Interface Protocols

4.1 HUB75 RGB LED Matrix Protocol

HUB75 is the de facto standard parallel interface for driving RGB LED matrix panels directly from a microcontroller or single-board computer. It uses a 16-pin IDC (2×8) connector and transmits pixel data serially, one row at a time, using a shift-register architecture. The protocol is synchronous: all signals are referenced to a common clock (CLK).

The HUB75 protocol drives *two* rows simultaneously per clock cycle (upper half and lower half of the panel are driven in parallel via separate R1/G1/B1 and R2/G2/B2 data lines). For a 32-row panel, the row select lines A, B, C, D address 16 pairs, yielding 16 multiplex steps per frame.

> **HUB75E VARIANT**
>
> HUB75**E** adds a fifth row-select line (E) to support 64-row panels (32 pairs). P5 modules commonly use HUB75E. The ATLAS HAL must detect the panel height and configure the row-select bus accordingly (4-bit for 32-row, 5-bit for 64-row).

4.2 Alternative Protocols

© ATLAS Project 2026

4.2.1 Artnet DMX (UDP/IP)

Artnet is an Ethernet-based protocol that encapsulates DMX512 data in UDP packets, transmitting pixel values over a standard network switch. It is supported by commercial LED controllers (e.g., Novastar, Colorlight) and is appropriate when the host computer and LED controller are connected via Ethernet rather than GPIO/USB. Artnet Universe 0, starting at DMX address 1, can carry 512 channels, sufficient for a 64×64 display as 8-bit RGB (512/3 = 170 pixels per universe; chained universes required for larger arrays).

4.2.2 Serial / UART

A simplified serial protocol over USB-to-UART or RS-232 is viable for custom or low-cost embedded controllers. A text-based or binary frame structure (see Section 5.3) can be transmitted from the Mac Mini at 115,200 baud or higher. This approach requires a custom firmware image on the controller board and is the least mature of the three protocol options. It is included for completeness and for scenarios where Ethernet and GPIO both present integration challenges.

4.3 Protocol Selection Criteria

Table 4-1 Protocol Selection Matrix

Criterion	HUB75 Direct	Artnet DMX	Serial/UART
Controller hardware cost	Low (RPi Zero 2W ~$20)	Medium ($150–$500)	Very low (<$10)
Max panel refresh rate	High (≥120 Hz at 128×64)	Medium (≥40 Hz)	Low (≥10 Hz at 115.2 kbps)
Cabling distance	Short (≤1 m flat ribbon)	Long (100 m Ethernet)	Medium (10–15 m USB)
Software complexity	Medium (bit-bang or rpi-rgb-led-matrix lib)	Low (UDP socket)	Medium (binary framing)
Commercial support	Community (well-documented)	Industry standard	Custom only
Recommended for ATLAS	**Primary choice**	Fallback / expansion	Prototype only

The recommended implementation for ATLAS hardware integration is **HUB75 direct-drive using a Raspberry Pi 4 or Pi Zero 2W** running the `rpi-rgb-led-matrix` open-source library. The Mac Mini connects to the Raspberry Pi over USB (acting as a USB-Serial bridge) or over Ethernet (local TCP socket), and the Pi drives the LED panels directly via GPIO. This architecture decouples the ATLAS application from the real-time pixel-clocking requirements of HUB75.

© ATLAS Project 2026

5. Signal Definitions

5.1 HUB75 Connector Pinout

The HUB75 interface uses a 16-pin IDC (2×8) connector. Table 5-1 defines the pin assignments for the standard HUB75 and HUB75E variants. All signal lines are 3.3 V or 5 V CMOS logic; the controller must match the panel's input voltage (check panel datasheet). Series termination resistors (33–47 Ω) on data and clock lines are recommended for cable runs exceeding 0.3 m.

Table 5-1 HUB75 / HUB75E Connector Pinout (2×8 IDC, numbered top-to-bottom, odd = left column)

Pin	Signal	Direction	Description	Logic Level
1	R1	Controller → Panel	Red data — upper half of panel	3.3 / 5 V
2	G1	Controller → Panel	Green data — upper half of panel	3.3 / 5 V
3	B1	Controller → Panel	Blue data — upper half of panel	3.3 / 5 V
4	GND	—	Signal ground (0 V reference)	0 V
5	R2	Controller → Panel	Red data — lower half of panel	3.3 / 5 V
6	G2	Controller → Panel	Green data — lower half of panel	3.3 / 5 V
7	B2	Controller → Panel	Blue data — lower half of panel	3.3 / 5 V
8	GND	—	Signal ground	0 V
9	A	Controller → Panel	Row select bit 0 (LSB)	3.3 / 5 V
10	B	Controller → Panel	Row select bit 1	3.3 / 5 V
11	C	Controller → Panel	Row select bit 2	3.3 / 5 V
12	D	Controller → Panel	Row select bit 3 (MSB for 32-row)	3.3 / 5 V
13	CLK	Controller → Panel	Shift register clock (rising edge active)	3.3 / 5 V

© ATLAS Project 2026

14	STB / LAT	Controller → Panel	Strobe / Latch: transfers shift register to output	3.3 / 5 V
15	OE	Controller → Panel	Output Enable (active LOW): enables LED drivers	3.3 / 5 V
16	E (HUB75E)	Controller → Panel	Row select bit 4 — required for 64-row panels (P5)	3.3 / 5 V

5.2 Timing Diagrams

Figure 5-1 illustrates the HUB75 single-row scan cycle. Each scan cycle clocks out one row of pixel data, latches it, and enables the LED drivers for the blanking period. The OE signal is used for PWM brightness control by modulating the enable duration.

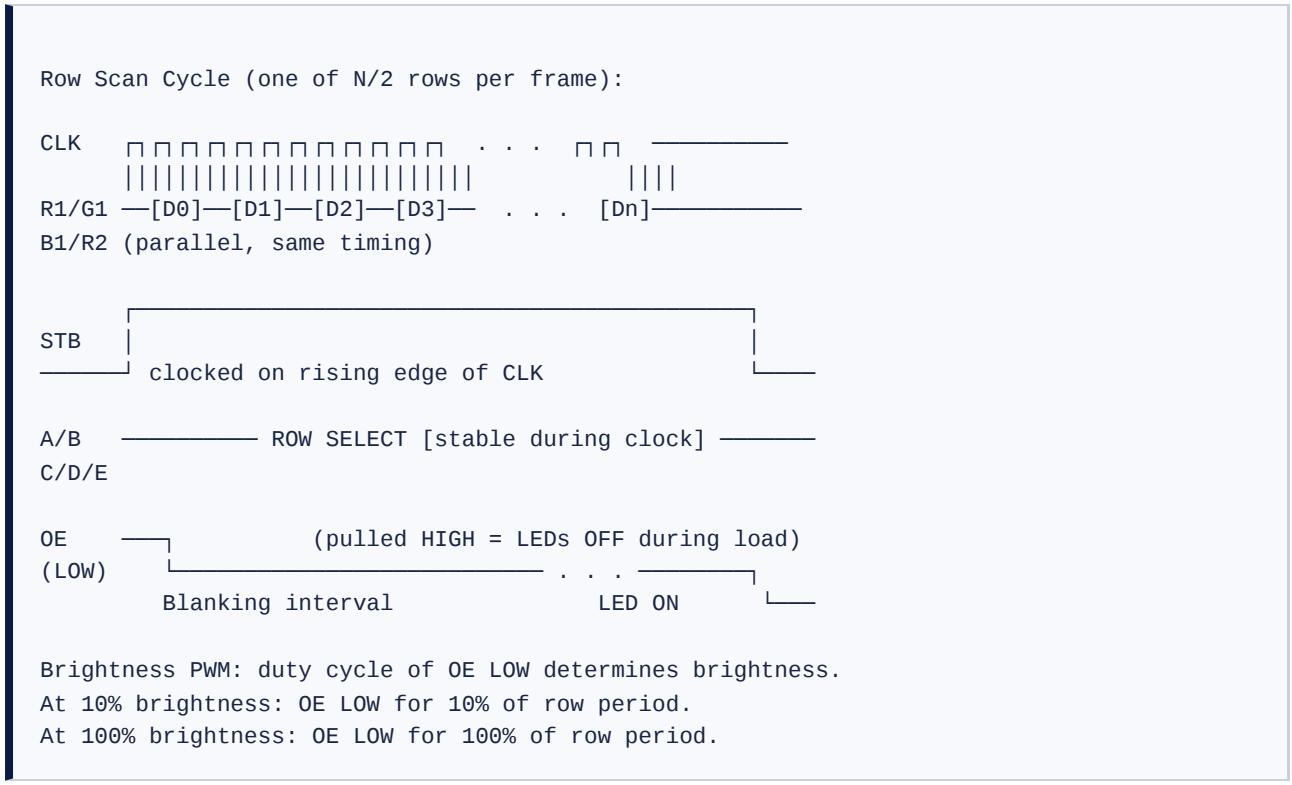


Figure 5-1 HUB75 Row Scan Timing — Single Row Cycle

Table 5-2 HUB75 Timing Parameters (Typical, 5 V CMOS)

Parameter	Symbol	Min	Typical	Max	Unit
Clock frequency	f_CLK	1	10	20	MHz
Clock high time	t_CH	20	50	—	ns
Clock low time	t_CL	20	50	—	ns
Data setup time (to CLK rise)	t_DS	5	—	—	ns
Data hold time (after CLK rise)	t_DH	5	—	—	ns
STB pulse width	t_STB	50	100	—	ns

© ATLAS Project 2026

Row select settling time	t_RS	100	—	—	ns
Frame refresh period (128×64, 60 Hz)	t_FR	—	16.7	—	ms

5.3 Data Frame Format

The ATLAS HAL represents a complete display frame as a two-dimensional byte array of dimensions `[height][width][3]`, where the innermost dimension contains the R, G, B byte values (8-bit per channel). Brightness scaling is applied by the HAL before transmission; the engine and model layers always operate in full-scale (0–255) colour.

```
# Frame buffer type definition (Python pseudo-specification)
FrameBuffer = List[List[Tuple[int, int, int]]]   # [row][col] = (R, G, B)

# Dimensions: frame[row][col] where
#   row in range(0, display_height)   # 0 = top
#   col in range(0, display_width)    # 0 = left
#   R, G, B in range(0, 256)          # full 8-bit colour, pre-brightness-scaled

# Example: set pixel at row=5, col=10 to full red
frame[5][10] = (255, 0, 0)

# Brightness application before HAL send:
scaled = tuple(int(ch * brightness / 100) for ch in (R, G, B))
```

5.4 Row Addressing for 6-Row Configuration

It is important to distinguish between the *logical* display rows defined by the ATLAS 6-row layout strategy (Section 2.3) and the *physical* row scan lines of the HUB75 panel. The 6-row layout refers to content regions defined in pixel coordinates; the HUB75 panel scans each physical pixel row independently. Figure 5-2 maps logical rows to physical pixel rows for the Full profile (128×64).

© ATLAS Project 2026

```
LOGICAL ROW → PHYSICAL PIXEL ROW MAPPING (Full Profile, 128×64 px)

Logical Row 1 (Local Time)          pixels Y:  0 –  9  (10 px)
Logical Row 2 (Heat Info)           pixels Y: 10 – 19  (10 px)
Logical Row 3 (Window Times)        pixels Y: 20 – 29  (10 px)
Logical Row 4 (Phase Label)         pixels Y: 33 – 42  (10 px, Y=35 origin + 7px font)
Logical Row 5 (Timer 2× scale)      pixels Y: 43 – 58  (16 px — 2× scaled 5×7 font)
Logical Row 6 (Status Band)         pixels Y: 62 – 63  ( 2 px solid fill)

HUB75 Physical Scan Pairs (64-row panel, 32 pairs via A/B/C/D/E):
  Pair  0: physical rows  0,  32  →  covers Logical Rows 1–2
  Pair  1: physical rows  1,  33  →  covers Logical Rows 1–2
  ...
  Pair  9: physical rows  9,  41  →  covers Logical Rows 1–4
  ...
  Pair 15: physical rows 15,  47  →  covers Logical Rows 4–5
  ...
  Pair 31: physical rows 31,  63  →  covers Logical Rows 5–6
```

Figure 5-2 Logical Row to Physical HUB75 Scan Row Mapping (128×64 Full Profile)

> **NOTE — NO SOFTWARE ROW MAPPING REQUIRED**
>
> The `rpi-rgb-led-matrix` library abstracts away the physical scan pairs; software writes to a 2D pixel coordinate space and the library handles interleaving. Direct register-level implementations must account for the pair interleaving shown in Figure 5-2.

6. Hardware Abstraction Layer (HAL)

The Hardware Abstraction Layer is the critical architectural boundary between the ATLAS application software and the physical display hardware. All hardware-specific code is contained within backend modules that implement the `DisplayBackend` abstract interface. No hardware-specific code shall appear in `engine.py`, `models.py`, `windows.py`, or `database.py`.

6.1 Abstract Interface Design

The `DisplayBackend` abstract base class defines the complete contract that every backend must fulfil. It consists of five mandatory methods. Any class that does not implement all five methods cannot be registered as a valid backend.

© ATLAS Project 2026

```
from abc import ABC, abstractmethod
from core.models import AtlasDisplayModel

class DisplayBackend(ABC):
    """
    ATLAS HAL — Abstract Display Backend (ATLAS-ICD-001 §6.1)
    All physical and virtual display implementations must inherit
    from this class and implement every abstract method.
    """

    @abstractmethod
    def initialize(self, config) -> bool:
        """
        Allocate hardware resources, open device handles, and configure
        the display for operation.
        :param config: DisplayConfig object (width, height, brightness, etc.)
        :return: True if initialisation succeeded; False on failure.
        """

    @abstractmethod
    def render(self, model: AtlasDisplayModel) -> None:
        """
        Receive an AtlasDisplayModel snapshot and render a complete frame
        to the display. Called once per engine tick (≈1 Hz).
        Must NOT block for more than 100 ms.
        """

    @abstractmethod
    def set_brightness(self, value: int) -> None:
        """
        Adjust display brightness in real time.
        :param value: Integer percentage, range 10–100 inclusive.
        Must take effect within 2 seconds without restarting the app.
        """

    @abstractmethod
    def clear(self) -> None:
        """
        Set all pixels to off (black). Called on graceful shutdown
        to blank the display before power-down.
        """

    @abstractmethod
    def shutdown(self) -> None:
        """
        Release all hardware resources. Called once on application exit.
        Must call clear() before releasing resources.
        """
```


© ATLAS Project 2026

6.2 Simulator Backend (Current Implementation)

The current system implements a `SimulatorBackend` that wraps the existing `AtlasLEDSimulator` Qt widget. This backend renders the 6-row layout using QPainter onto a Qt window, with circular LED pixels and a 5×7 bitmap font. It is fully functional and is the production backend for all software-only deployments.

Table 6-1 Simulator Backend Characteristics

Property	Value
Backend identifier	`"simulator"`
Module	`led_simulator.py` — `AtlasLEDSimulator`
Rendering engine	Qt6 QPainter (PySide6)
Font	5×7 bitmap, `AtlasBitmapFont.FONT_5x7`
LED pixel style	Filled ellipse, radius = `led_scale − 1`
Brightness scaling	Linear per-channel multiply: `ch × brightness / 100`
Update rate	1 Hz (driven by engine tick)
Adaptive layout	Full (cols > 96) / Compact (cols ≤ 96) auto-selected
Physical hardware required	None — software only

6.3 Physical Hardware Backend (Future)

The `HUB75Backend` will be implemented to drive physical LED panels via a Raspberry Pi controller. The Mac Mini communicates with the Raspberry Pi over a local TCP socket (preferred) or USB-Serial link. The Raspberry Pi runs a lightweight Python daemon (`atlas_display_daemon.py`) that accepts `AtlasDisplayModel` JSON objects, renders the 6-row frame using the `rpi-rgb-led-matrix` Python bindings, and writes directly to the HUB75 GPIO bus.

© ATLAS Project 2026

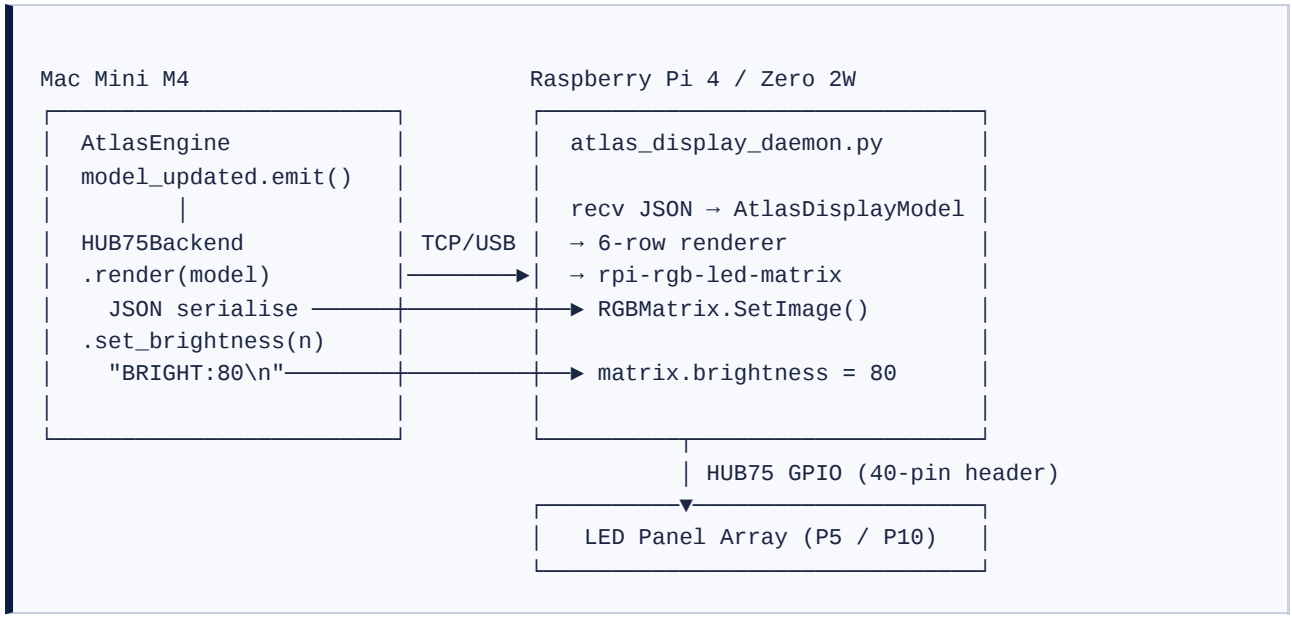


Figure 6-1 HUB75 Physical Backend Architecture

The `ArtnetBackend` is an alternative future backend for installations using a commercial LED controller connected to the venue Ethernet infrastructure. It packages the rendered frame buffer as Artnet universe packets and transmits them via UDP broadcast to the controller's IP address.

6.4 Backend Selection Mechanism

The backend is selected at application startup from the `display_backend` key in the ATLAS settings database. The backend factory module returns a fully configured backend object ready for `initialize()` to be called. No restart is required to switch brightness; a restart is required to switch backend type.

```
# backend_factory.py  (ATLAS-ICD-001 §6.4)
BACKEND_REGISTRY = {
    "simulator": SimulatorBackend,
    "hub75":     HUB75Backend,      # Future
    "artnet":    ArtnetBackend,     # Future
    "serial":    SerialBackend,     # Prototype
}

def create_display_backend(backend_type: str) -> DisplayBackend:
    cls = BACKEND_REGISTRY.get(backend_type)
    if cls is None:
        print(f"[WARN] Unknown backend '{backend_type}', falling back to simulator")
        cls = SimulatorBackend
    return cls()
```

Table 6-2 Backend Selection — Database Setting Values

Setting Value	Backend	Status	Notes
`"simulator"`	SimulatorBackend	CURRENT	Qt6 window, no hardware required

© ATLAS Project 2026

`"hub75"`	HUB75Backend	FUTURE	Raspberry Pi + rpi-rgb-led-matrix
`"artnet"`	ArtnetBackend	FUTURE	Commercial controller, UDP/IP
`"serial"`	SerialBackend	TBD	USB-Serial embedded controller

7. Integration Requirements

7.1 Power Requirements

LED panel power consumption must be assessed for the specific panel model and array size selected. Table 7-1 provides indicative power budgets for the recommended configurations. A margin of at least 25% shall be applied to all power supply ratings to account for cold-start inrush current and peak brightness operation.

Table 7-1 Indicative Power Budget by Configuration

Configuration	Panel Count	Avg Power	Peak Power	Min PSU Rating	Supply Voltage
P5 Compact (64×64)	2 modules	35 W	90 W	5 V / 25 A	5 V DC
P5 Full (128×64)	4 modules	70 W	180 W	5 V / 50 A	5 V DC
P10 Compact (64×64)	8 modules	60 W	150 W	5 V / 40 A	5 V DC
P10 Full (128×64)	16 modules	120 W	300 W	5 V / 80 A	5 V DC

WARNING — POWER SEQUENCING

Always power the LED panels **before** asserting HUB75 data signals. Applying data signals to unpowered panels may damage input protection diodes. The ATLAS shutdown procedure must clear (blank) the display before cutting panel power.

7.2 Cabling and Connectors

- **Panel power cable:** 5 V / 0 V on 14 AWG (≥2.5 mm²) minimum for runs up to 1 m. Voltage drop shall not exceed 0.2 V at peak load. Use star-topology power distribution from the PSU to each panel module.
- **HUB75 data cable:** Standard 16-pin IDC flat ribbon cable. Maximum recommended length between controller and first panel: 0.5 m. Between daisy-chained panels: use panel's built-in pass-through connector; do not extend with additional cable.
- **Controller-to-host cable (Raspberry Pi option):** Standard USB-A to USB-Micro or USB-C, maximum 2 m, or shielded Cat-5e Ethernet for TCP socket connection, maximum 100 m.

© ATLAS Project 2026

- **Grounding:** The signal ground of the HUB75 connector (pins 4 and 8) must be bonded to the PSU negative rail and to the panel chassis. A single-point ground at the PSU is preferred.

7.3 Environmental Considerations

Table 7-2 Environmental Operating Requirements

Parameter	P5 Indoor	P10 Outdoor
Operating temperature	-10 °C to +45 °C	-20 °C to +55 °C
Storage temperature	-20 °C to +65 °C	-40 °C to +70 °C
Relative humidity (operating)	10–80% non-condensing	10–95% non-condensing
IP rating	IP31 (drip-proof)	IP65 (dust/jet water proof)
Wind load (outdoor)	N/A	Must be rated for site-specific wind loading; consult structural engineer for permanent mounting
Vibration (transport)	Packing as per manufacturer	Packing as per manufacturer

7.4 Safety and Compliance

The following electrical safety and EMC requirements apply to the complete ATLAS display installation:

- All power supplies must carry an applicable safety certification for the jurisdiction of use (e.g., SAA/RCM for Australia/NZ, CE for EU, UL for USA/Canada).
- The installation must comply with AS/NZS 3000 wiring rules (Australia/NZ) or the applicable national electrical code.
- LED display panels are Class 1 equipment; the panel chassis shall be protectively earthed (green/yellow conductor, minimum 1.5 mm^2).
- EMC emissions from the HUB75 flat ribbon cable can be significant. Cable shall be routed away from RF-sensitive equipment (e.g., radio receivers, GPS units). Ferrite clamps on the ribbon cable at both the controller and panel ends are recommended for P10 outdoor installations.
- A residual current device (RCD/GFCI) of 30 mA or less shall protect the AC supply circuit feeding the display PSU, per AS/NZS 3000:2018 requirements for outdoor electrical equipment.

8. Testing and Validation

© ATLAS Project 2026

8.1 Interface Verification Procedures

The following verification tests shall be conducted upon installation of physical hardware and after any modification to the display system or its software. Tests are listed in the order they shall be executed; any failure halts the sequence until the defect is rectified.

Table 8-1 Interface Verification Test Sequence

Test ID	Name	Procedure	Pass Criterion
IVT-01	Power-On Self-Test	Apply mains power; observe display for 10 s	All pixels illuminate in sequence (panel self-test); no flickering; no dead rows or columns
IVT-02	HAL Backend Initialisation	Launch ATLAS application with target backend configured	Application starts without error; operator console displays; no "Failed to initialize display backend" message
IVT-03	Blank Frame Render	Call `display_backend.clear()` via test harness	All pixels extinguish within 1 second
IVT-04	Full-Colour Fill	Send solid Red, Green, Blue frames sequentially	Each colour fills all pixels uniformly; no dead LEDs visible at normal brightness
IVT-05	6-Row Layout Render	Inject a known `AtlasDisplayModel` with all fields populated; render to display	All six rows render in correct positions; correct colours; no truncation or overflow at panel edges
IVT-06	Brightness Control	Sweep brightness from 10% to 100% in 10% steps; hold 2 s per step	Display luminance increases monotonically; no colour shift; no flicker; response time ≤ 2 s per step
IVT-07	1 Hz Update Rate	Run ATLAS with an active heat for 60 s; time-stamp each render call	All 60 render calls complete; interval between renders 950–1,050 ms; no dropped frames
IVT-08	Graceful Shutdown	Click "EXIT TO macOS"; observe display	Display blanks completely before application window closes; no residual pixel state
IVT-09	Phase Transition Display	Schedule a heat with track_open 90 s in the future; observe display through all phases	Phase label and colour change correctly at each transition: Yellow→Green→Red; no rendering artefacts

© ATLAS Project 2026

IVT-10	Compact / Full Mode Switch	Configure 64×64 profile; restart; verify compact mode activates; switch to 128×64; restart; verify full mode	Correct mode selected for each resolution; abbreviations applied in compact mode

8.2 Acceptance Criteria

The ATLAS display hardware integration is accepted when all ten IVT tests pass with zero failures in a single test run. A partial pass (nine of ten) is not acceptable; all tests are mandatory. The test results shall be documented on an IVT sign-off sheet referencing this document (ATLAS-ICD-001 Rev 1.0) and retained as part of the project commissioning record.

The following quantitative performance requirements constitute the minimum acceptance threshold for physical hardware:

- Frame render latency from model receipt to first pixel visible: ≤ 100 ms
- Brightness adjustment response time: ≤ 2 seconds
- Panel refresh rate (hardware scan rate): ≥ 60 Hz (P10), ≥ 120 Hz (P5)
- Dead pixel count: 0 (zero tolerance; any dead pixel requires panel replacement)
- Operational uptime during a six-hour competition day: ≥ 99.5% (maximum 108 s of unplanned outage)

Appendix A — Glossary of Terms

Artnet DMX
: An industry-standard protocol encapsulating DMX512 lighting control data in UDP/IP packets for transmission over Ethernet networks.

AtlasDisplayModel
: The Pydantic data model class in `models.py` that carries all display-relevant state from the AtlasEngine to the HAL backend once per second.

AtlasEngine
: The core timing state machine in `engine.py`. Maintains phase P0–P4 transitions, UTC-referenced countdown timers, and auto-activation of heats.

Compact Mode
: The six-row display layout profile selected when display width ≤ 96 px. Uses abbreviated phase labels and single-scale font throughout.

DisplayBackend
: The abstract base class (ATLAS HAL) that all display implementations must inherit. Defines the `initialize`, `render`, `set_brightness`, `clear`, and `shutdown` interface contract.

Full Mode
: The six-row display layout profile for display width > 96 px (e.g., 128×64). Uses 2× scaled primary timer font and inline track window times.

© ATLAS Project 2026

HAL

Hardware Abstraction Layer. The architectural boundary in ATLAS that isolates application software from hardware-specific display drivers.

HUB75

A 16-pin IDC parallel interface standard used to drive RGB LED matrix panels. The primary approved protocol for ATLAS physical hardware integration.

HUB75E

An extended variant of HUB75 adding a fifth row-select line (E) to support panels with 64 or more rows, such as P5 indoor panels.

ICD

Interface Control Document. A formal engineering document defining the interface between two subsystems or components.

IDC

Insulation-Displacement Connector. A cable connector type that terminates multi-conductor flat ribbon cables by piercing the insulation.

OE

Output Enable. An active-LOW signal on HUB75 that gates the LED driver outputs. PWM modulation of OE controls display brightness.

P5

A 5 mm pixel pitch LED panel type, typically SMD-packaged, recommended for indoor or short-viewing-distance applications.

P10

A 10 mm pixel pitch LED panel type, typically available in outdoor-rated (IP65) configurations, recommended for flight line and grandstand deployments.

Phase (P0–P4)

The five operational states of the ATLAS timing engine relative to the current heat's track_open, track_close, and heat_end timestamps.

PSU

Power Supply Unit. Provides regulated 5 V DC power to LED panel modules.

PWM

Pulse-Width Modulation. Technique used to control LED brightness by rapidly switching power at varying duty cycles.

RCD

Residual Current Device (also RCCB or GFCI). A safety device that disconnects the AC supply if it detects current leakage to earth.

rpi-rgb-led-matrix

An open-source C/Python library by Henner Zeller for driving HUB75 LED panels directly from Raspberry Pi GPIO.

STB / LAT

 © ATLAS Project 2026

Strobe / Latch. A HUB75 signal that transfers the contents of the shift register to the LED driver output latches, making a new row of pixels visible.

Appendix B — Reference Documents

Table B-1 Complete Reference Document List

Reference ID	Title	Source / Author	Version / Date
ATLAS-SDS-001	ATLAS Software Design Specification	ATLAS Engineering	Current (2026)
ATLAS-SHD-001	ATLAS System Handover Dossier	ATLAS Engineering	2026-07-03
ATLAS-TU-002	ATLAS Technical Update	ATLAS Engineering	2026-07-02
HUB75-SPEC	HUB75 RGB LED Matrix Interface Specification	Community / Henner Zeller	Rev D
RPI-RGB-LED	rpi-rgb-led-matrix Library Documentation	Henner Zeller (GitHub)	Latest
IEC 60950-1	Information Technology Equipment — Safety — Part 1: General Requirements	IEC	Edition 2.2
CISPR 32	Multimedia Equipment — Electromagnetic Disturbances — Requirements for Emission	CISPR / IEC	Edition 3
AS/NZS 3000	Wiring Rules — Electrical Installations of Buildings	Standards Australia	2018
AS/NZS 3820	Essential Requirements for Electrical Equipment	Standards Australia	1998
ARTNET-SPEC	Art-Net 4 Protocol Specification	Artistic Licence (UK) Ltd	Revision 1.4dd

Appendix C — Revision History

Table C-1 Document Revision History

Revision	Date	Author	Changes
1.0	2026-07-03	ATLAS Engineering / Abacus AI Agent	Initial document. Covers modular ICD framework: 6-row display strategy, P5/P10 panel specifications, HUB75 protocol definitions, HAL abstract interface design, simulator backend documentation, and provisional integration requirements. Hardware-specific electrical values (exact panel model, controller board part number) to be finalised in Rev 1.1 after procurement.

© ATLAS Project 2026

1.1	TBD	TBD	Planned: Insert specific panel model and controller board; complete connector part numbers; add commissioning test results for IVT-01 through IVT-10.
1.2	TBD	TBD	Planned: HUB75Backend implementation details; Artnet backend evaluation results; field test data from competition deployment.

— End of ATLAS-ICD-001 Interface Control Document, Revision 1.0 —

Issued 3 July 2026 | ATLAS Master Flightline Clock Project | UNCONTROLLED WHEN PRINTED

© ATLAS Project 2026